OVERVIEW / ■■

液态玻璃，是一条可以顺着读懂的渲染链。

它由五步组成：先确定背景，再裁出组件所在区域，建立圆角光学场，计算折射与材质，最后把结果保存为稳定画面。文字和交互始终留在普通 DOM 上层。



READING RULE

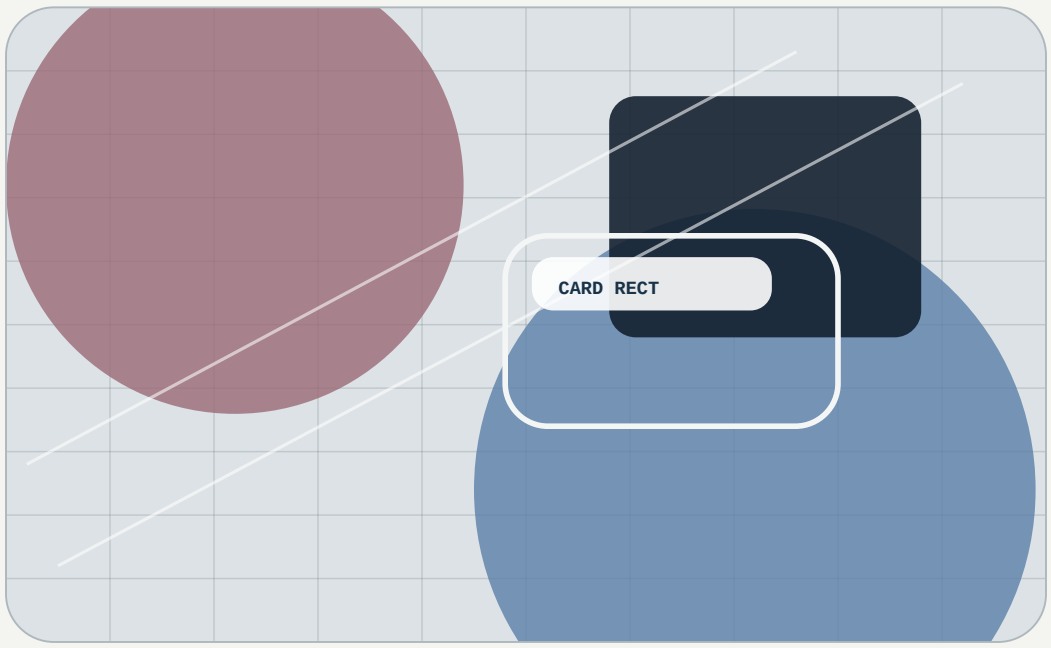
从背景到快照是一条数据链；从 Canvas 到文字是一层合成关系。

01 / BACKGROUND SAMPLING

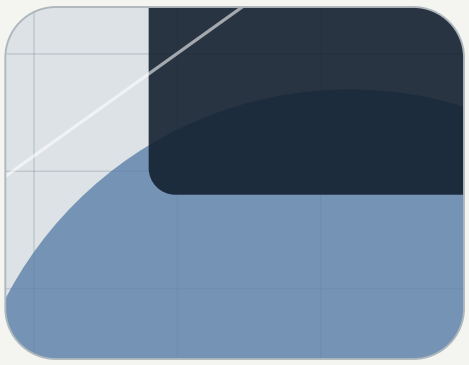
先找准卡片在背景里的位置。

同一张背景先铺满视口。每块玻璃读取自己的矩形位置，再从背景中裁出完全对应的那一块；这样折射后的纹理才不会错位。

1. FULL VIEWPORT



2. LOCAL TEXTURE



同一位置
同一纹理

1 MEASURE

读取卡片的 rect：left、top、width、height。

2 CROP

按 cover 后的背景比例换算局部偏移。

3 UPLOAD

把局部 Canvas 上传为当前卡片纹理。

LOCAL CROP FROM THE CONTROLLED BACKGROUND SOURCE

```
01  const rect = card.getBoundingClientRect();
02  const scale = Math.max(vw / iw, vh / ih);
03  const dx = (vw - iw * scale) / 2 - rect.left;
04  const dy = (vh - ih * scale) / 2 - rect.top;
05  ctx.drawImage(image, dx, dy, iw * scale, ih * scale);
```

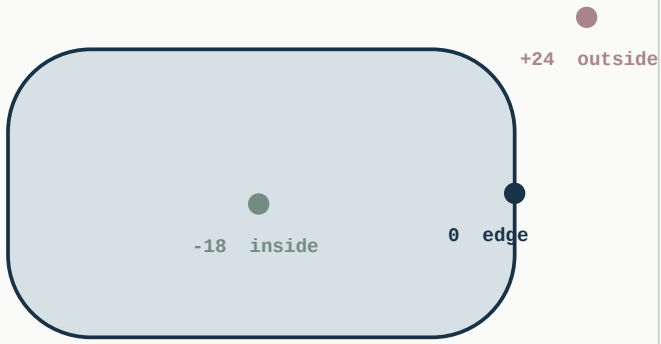
边界：这里重建的是受控背景源（默认渐变或本地压缩后的壁纸 data URL）与主题蒙层。普通 DOM、视频和另一层玻璃不会自动进入纹理。

02 / ROUNDED GEOMETRY

光学场必须知道真实圆角在哪里。

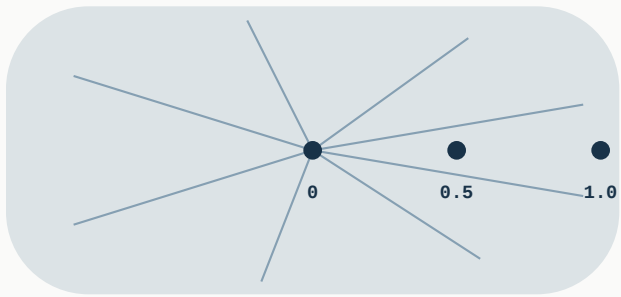
两个量解决两件事：roundBox() 判断像素离圆角边界有多远；radialBoundary() 把不同方向的边界都归一为 1。

A. SIGNED DISTANCE



负值在内部，0 是边界，正值在外部。它负责裁出真正的圆角轮廓。

B. NORMALIZED FIELD



无论射线撞到直边还是圆角，边界都变成 1；宽卡片不会被当成长圆镜片。

ROUNDED RECTANGLE SDF

```
vec2 q = abs(point-u_res*.5)
  -(u_res*.5-vec2(u_corner));
float sd = length(max(q,0.))
  + min(max(q.x,q.y),0.) - u_corner;
```

FIELD COORDINATES

```
vec2 rel = px - u_res * .5;
vec2 p = rel / radialBoundary(rel);
float rr = clamp(length(p), 0., 1.);
// center = 0, exact boundary = 1
```

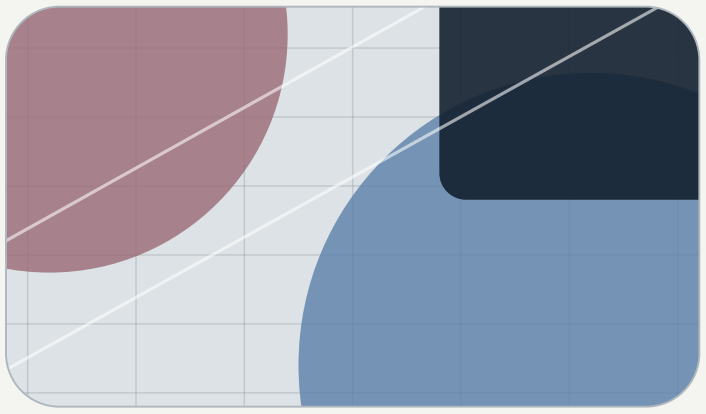
先用 SDF 得到形状，再用归一化场描述“离边缘还有多远”。后面的折射、暗边和高光都复用这两个结果。

03 / REFRACTION FIELD

折射，本质上是换一个位置读取背景。

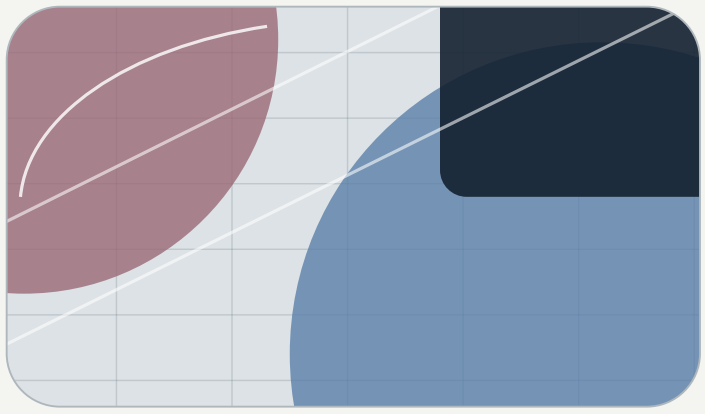
每个像素先得到归一化位置 p，再由整体透镜和低频波动计算位移 shift。最终不是给像素涂颜色，而是去背景纹理的邻近位置重新取色。

BEFORE



SHIFT UV

AFTER / REFRACTED



p

归一化位置

rr

中心到边缘

sh + wave

透镜与波动

refrUv

新的采样坐标

CONDENSED FROM THE PRODUCTION SHADER

```
01    float edgeGain = smoothstep(0.05, 0.97, rr);
02    float strength = (0.052 + 0.12 * edgeGain)
03      * u_refract * u_lDistort;
04    vec2 shift = p * sh + interiorWave * strength;
05    vec2 refrUv = v_uv + vec2(shift.x/u_res.x, -shift.y/u_res.y);
06    vec3 refr = backdrop(refrUv, u_blur * u_lBlur);
```

CURRENT TUNING

26	0	0	25	13	15
REFRACT	TINT	BLUR	EDGE	SHINE	WOBBLE

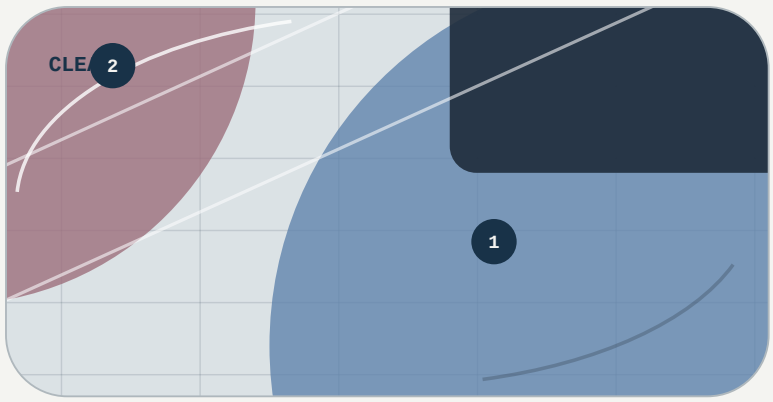
这些是 Shader 调节刻度，不是物理单位。

04 / MATERIAL AND LIGHT

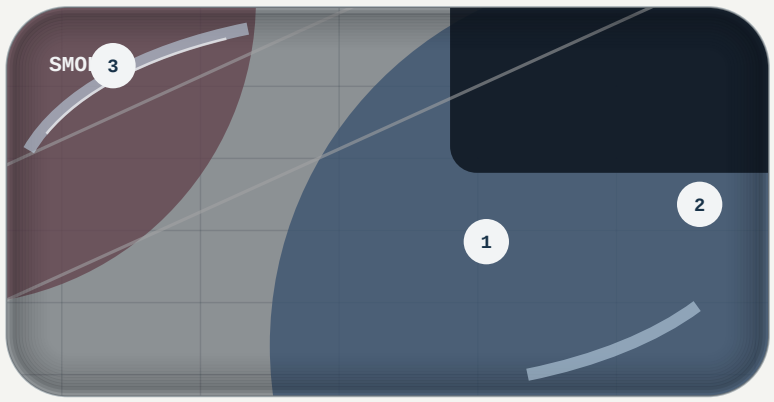
光学相同，透射方式可以完全不同。

透明与烟灰材质共用同一套折射坐标。区别在于背景透过多少、边缘吸收多少光，以及亮背景下要不要压低整圈白边。

SAME SOURCE TEXTURE / ■■■■■■



1. 背景细节保持清晰
2. 只保留方向性高光



1. 主体透射降低
2. 内侧暗边吸光
3. 边缘留下清透折射带

LUMINANCE ADAPTATION

BRIGHT BACKDROP

暗边更窄，均匀白环被压低

DARK BACKDROP

保留更多吸光厚度与局部高光

ADAPTIVE SMOKE AND DIRECTIONAL RIM

```
01    float lum = dot(color, vec3(0.2126, 0.7152, 0.0722));
02    float bright = smoothstep(0.42, 0.84, lum);
03    float smokeWidth = mix(7.2, 4.25, bright);
04    float edgeLoss = mix(0.27, 0.085, bright);
05    float hairline = exp(-pow((signedBorder + .46) / .43, 2.));
```

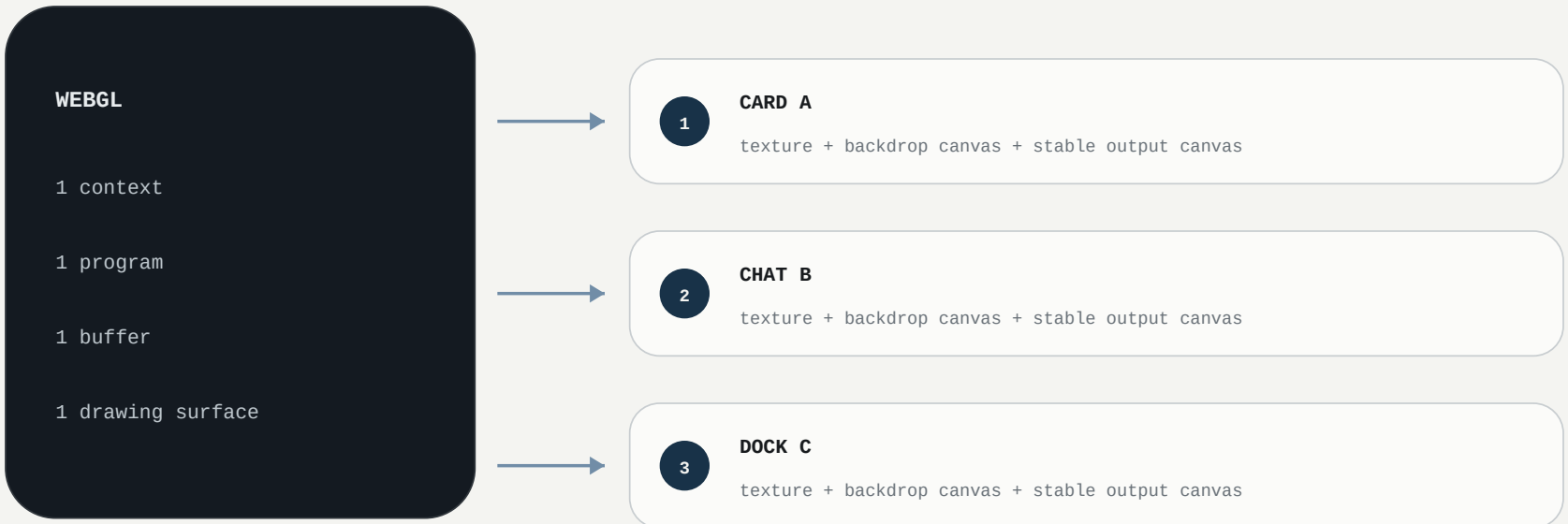
判断依据是实际采样到的背景亮度，不是日间/夜间标签。这样同一张壁纸里的亮区和暗区也能分别得到合适边缘。

05 / SHARED RENDERER

一个 WebGL 渲染器，许多独立快照。

玻璃引擎只创建一个 WebGL context。可见组件依次借用这块共享表面完成绘制，然后立即把像素复制到自己的 2D Canvas。

ONE SHARED GPU SURFACE



RENDER ONCE, KEEP A COMPONENT-OWNED SNAPSHOT

```
gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);
// Copy before the shared drawing buffer is discarded.
output.clearRect(0, 0, width, height);
output.drawImage(renderSurface, 0, 0);
// Normal DOM text remains above the output canvas.
```

WHEN A CARD LEAVES THE VIEWPORT

GPU TEXTURE	2D SNAPSHOT
释放	继续显示；context 丢失时旧画面仍在

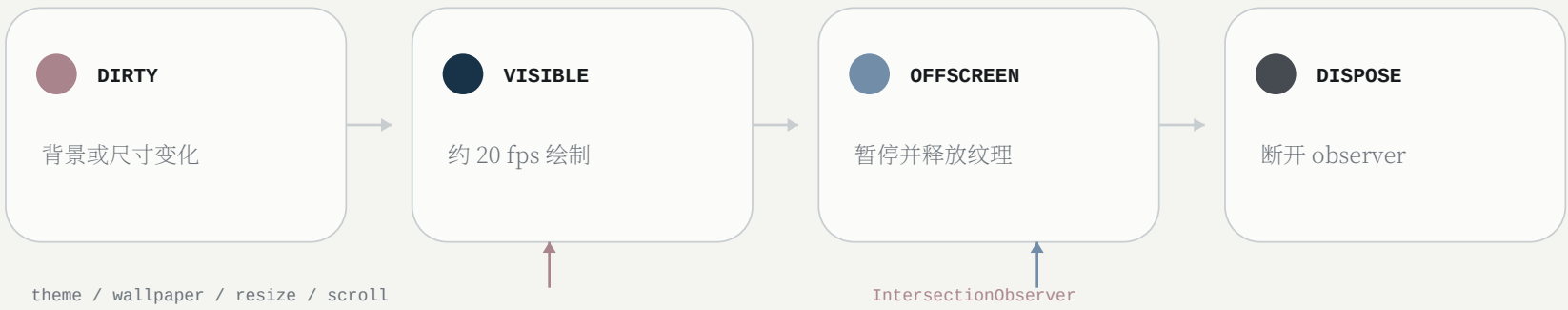
这里的“1 个 context”只指液态玻璃引擎。

06 / LIFECYCLE AND REUSE

动画一直存在，昂贵工作只在必要时发生。

背景纹理更新和材质动画分开管理。滚动、尺寸、主题或壁纸变化只把组件标为 dirty；可见组件按帧率上限绘制，离屏后保留快照并释放 GPU 纹理。

STATE FLOW



ACTUAL FRAME POLICY, CONDENSED

```
01    if (disposed || !visible || document.hidden || w < 5) return;
02    if (reduced) { if (dirty || generationChanged) draw(0); return; }
03    if (ms - last < 48) return;        // max about 20 fps
04    last = ms;
05    draw(ms * 0.001);                  // redraws texture only if dirty
```

CURRENT BUDGETS

<= 1.5	~ 20	1 engine	2.25 px	CSS blur
DPR	VISIBLE FPS	GL CONTEXT	HERO BLUR	FALLBACK

PUBLIC SCOPE

公开内容只包含光学思路、技术图与经过压缩的实现片段。真实应用、私人背景、聊天逻辑和个人数据不在其中。

UPSTREAM: VII-Cae/hyalite--liquid-glass (MIT) / everett745/liquid-glass (NO LICENCE DECLARED)

PROJECT: github.com/yanceydaisy/liquid-glass-note